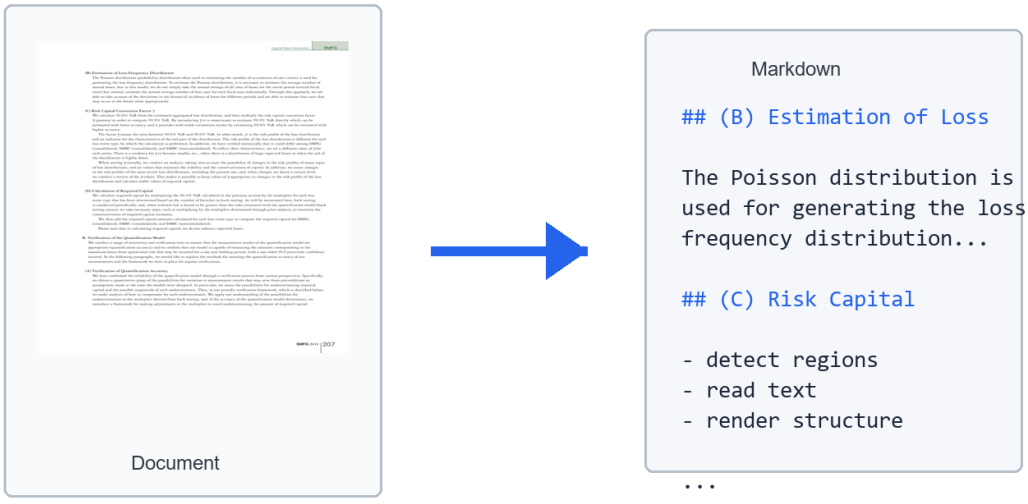


Document-to-Markdown Reconstruction

using YOLO Layout Detection and PP-OCRv6 Recognition

CAP 6665

Project 1

Connor Austin

June 22, 2026

1) Description

Description

This project builds a computer vision pipeline that converts scanned or image-based document pages into structured Markdown. The goal is not only to read text from a page, but to preserve enough document structure that the output remains useful for archival, search, indexing, and downstream text processing.

A good document reconstruction system must satisfy four practical criteria:

1. Semantic regions such as titles, section headers, paragraphs, lists, tables, pictures, captions, page headers, and footers should be detected correctly.
2. The detected regions should be ordered in the same sequence a human reader would follow.
3. OCR should read each text-bearing region accurately enough to preserve the meaning of the source page.
4. Markdown rendering should preserve document structure with headings, paragraphs, lists, table approximations, figures, captions, and page metadata.

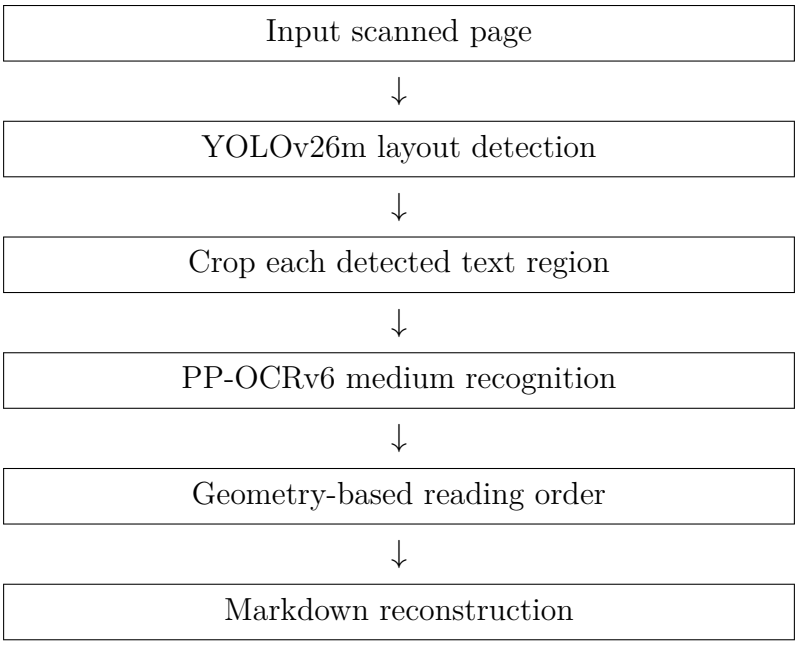
The implemented system uses YOLO for document layout detection and PP-OCRv6 medium recognition for text extraction. YOLO is responsible for finding semantic page regions. OCR is only applied after those regions are cropped, which keeps detection and recognition as separate, testable stages. The final stages sort regions geometrically and convert them into Markdown.

Dataset and Classes

The detector is trained on a DocLayNet-derived document layout dataset converted into YOLO format. The class set contains eleven document region types: Caption, Footnote, Formula, List-item, Page-footer, Page-header, Picture, Section-header, Table, Text, and Title. The data conversion scripts create separate detection and segmentation datasets so that both rectangular layout detection and mask-based experiments can be trained from the same source annotations.

Final Pipeline

The final pipeline used for the demo is:



The repository packages the final YOLO checkpoints and the OCR recognition model so the grader can clone the repository, run the included setup script, and start the demo without downloading model weights or using a Hugging Face token.

2) Results and Analysis

Detection Results

The strongest packaged model is the YOLOv26m detection checkpoint trained for 20 epochs at 1024 pixel image size, batch size 8, using 20% of the training data. Validation was run separately on the validation split. This model is used by default in the demo because it gives the best balance of layout quality, speed, and OCR compatibility.

Model	Precision	Recall	mAP50	mAP50-95
YOLOv26m detection final	0.869	0.825	0.884	0.724
YOLOv26m detection prototype	0.552	0.481	0.527	0.338
YOLOv26m segmentation prototype, boxes	0.537	0.501	0.532	0.355
YOLOv26m segmentation prototype, masks	0.527	0.494	0.519	0.345

The final detection model substantially outperformed the early detection and segmentation prototypes. Segmentation remains useful as an optional path when masks are needed, but for this project the rectangle detector was the better default because it gave stronger validation metrics and simpler OCR crops.

Analysis

The final detector performs best on frequent structural classes such as Text, Table, Page-footer, Caption, Picture, and Section-header. The precision-recall curve shows strong overall behavior with all-class mAP50 of 0.884. The normalized confusion matrix shows that the main remaining mistakes are between visually similar document elements, especially text-like regions and background or header/footer material.

OCR quality depends heavily on the crop passed to PP-OCRv6. Because the packaged Paddle model is a recognition model rather than a full text detector, the pipeline splits YOLO crops into lines and chunks before recognition. This works well for normal paragraphs and headings, but special symbols, low-resolution text, and table ruling lines can still reduce OCR quality. The implementation therefore removes long table ruling lines before OCR splitting and reconstructs tables from OCR line geometry when possible.

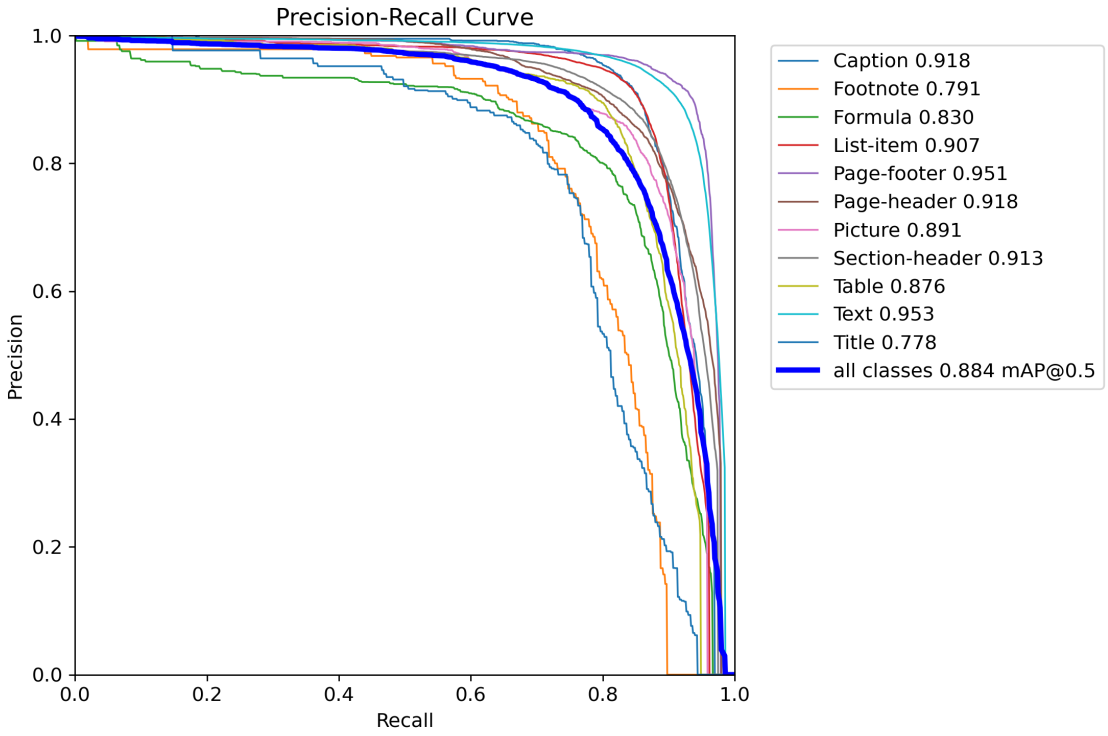


Figure 1: Precision-recall curve for the final YOLOv26m detection model.

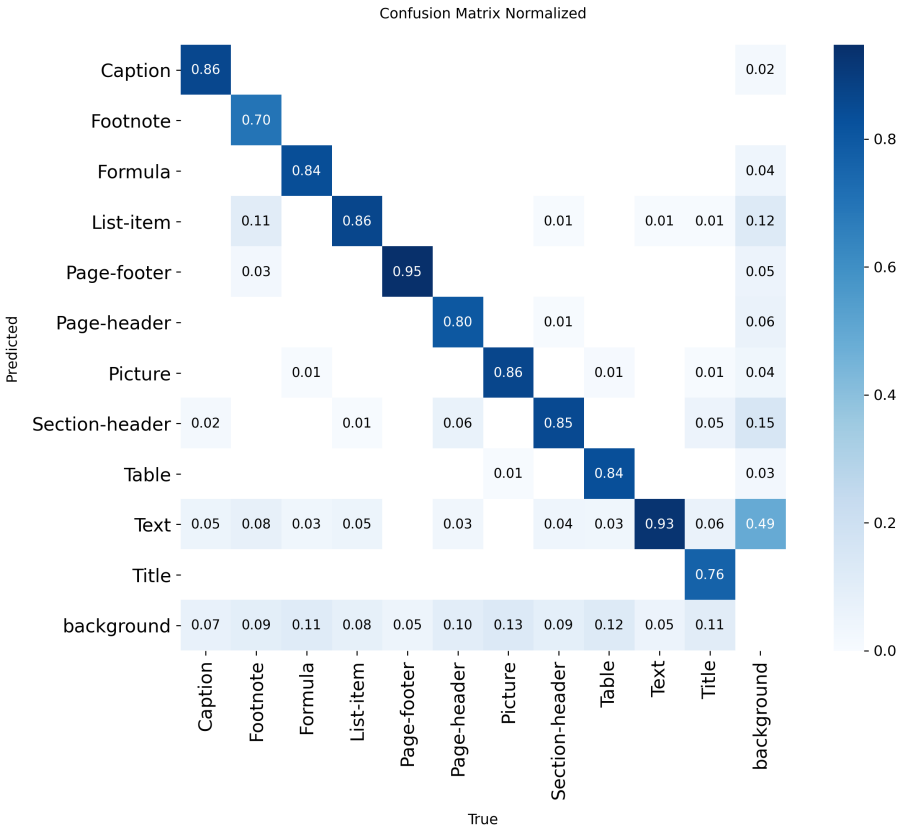
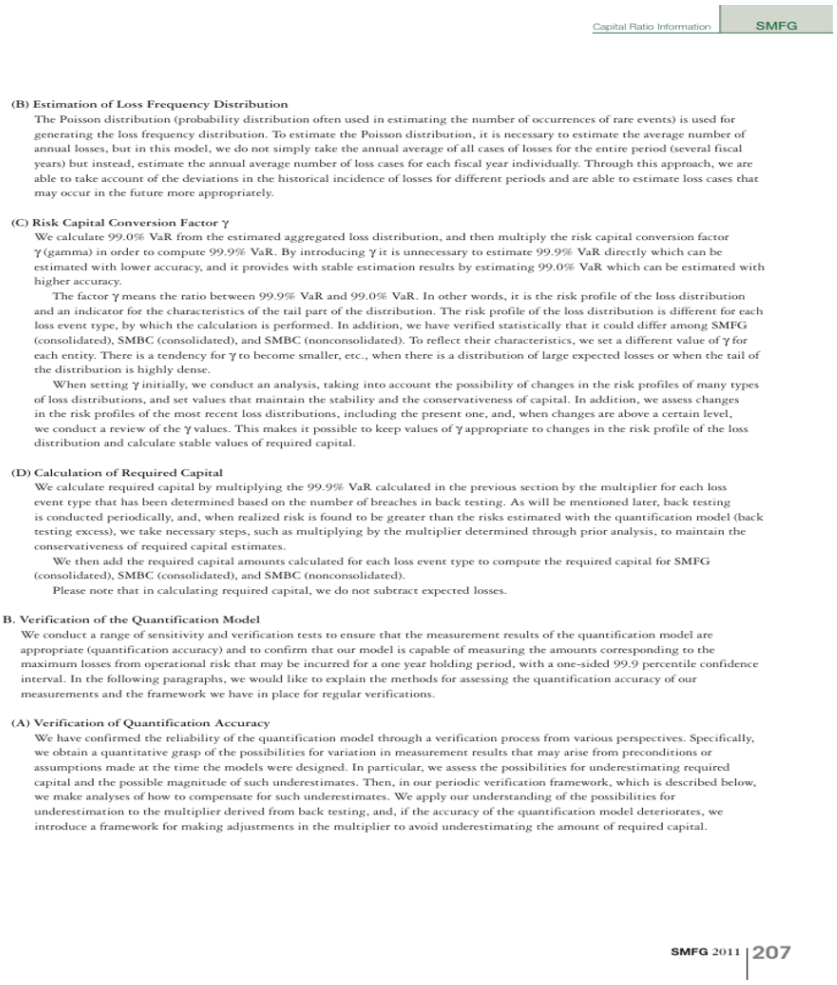


Figure 2: Normalized confusion matrix for the final detection validation run.

3) Visual Results

Input Page

The bundled demo page is a scanned financial report page with page headers, section headers, dense paragraphs, and a page footer. This page was kept as the single demo sample because it produces the cleanest end-to-end reconstruction and gives a representative mix of document region types.



(B) Estimation of Loss Frequency Distribution

The Poisson distribution (probability distribution often used in estimating the number of occurrences of rare events) is used for generating the loss frequency distribution. To estimate the Poisson distribution, it is necessary to estimate the average number of annual losses, but in this model, we do not simply take the annual average of all cases of losses for the entire period (several fiscal years) but instead, estimate the annual average number of loss cases for each fiscal year individually. Through this approach, we are able to take account of the deviations in the historical incidence of losses for different periods and are able to estimate loss cases that may occur in the future more appropriately.

(C) Risk Capital Conversion Factor γ

We calculate 99.0% VaR from the estimated aggregated loss distribution, and then multiply the risk capital conversion factor γ (gamma) in order to compute 99.9% VaR. By introducing γ it is unnecessary to estimate 99.9% VaR directly which can be estimated with lower accuracy, and it provides with stable estimation results by estimating 99.0% VaR which can be estimated with higher accuracy.

The factor γ means the ratio between 99.9% VaR and 99.0% VaR. In other words, it is the risk profile of the loss distribution and an indicator for the characteristics of the tail part of the distribution. The risk profile of the loss distribution is different for each loss event type, by which the calculation is performed. In addition, we have verified statistically that it could differ among SMFG (consolidated), SMBC (consolidated), and SMBC (nonconsolidated). To reflect their characteristics, we set a different value of γ for each entity. There is a tendency for γ to become smaller, etc., when there is a distribution of large expected losses or when the tail of the distribution is highly dense.

When setting γ initially, we conduct an analysis, taking into account the possibility of changes in the risk profiles of many types of loss distributions, and set values that maintain the stability and the conservativeness of capital. In addition, we assess changes in the risk profiles of the most recent loss distributions, including the present one, and, when changes are above a certain level, we conduct a review of the γ values. This makes it possible to keep values of γ appropriate to changes in the risk profile of the loss distribution and calculate stable values of required capital.

(D) Calculation of Required Capital

We calculate required capital by multiplying the 99.9% VaR calculated in the previous section by the multiplier for each loss event type that has been determined based on the number of breaches in back testing. As will be mentioned later, back testing is conducted periodically, and, when realized risk is found to be greater than the risks estimated with the quantification model (back testing excess), we take necessary steps, such as multiplying by the multiplier determined through prior analysis, to maintain the conservativeness of required capital estimates.

We then add the required capital amounts calculated for each loss event type to compute the required capital for SMFG (consolidated), SMBC (consolidated), and SMBC (nonconsolidated).

Please note that in calculating required capital, we do not subtract expected losses.

B. Verification of the Quantification Model

We conduct a range of sensitivity and verification tests to ensure that the measurement results of the quantification model are appropriate (quantification accuracy) and to confirm that our model is capable of measuring the amounts corresponding to the maximum losses from operational risk that may be incurred for a one year holding period, with a one-sided 99.9 percentile confidence interval. In the following paragraphs, we would like to explain the methods for assessing the quantification accuracy of our measurements and the framework we have in place for regular verifications.

(A) Verification of Quantification Accuracy

We have confirmed the reliability of the quantification model through a verification process from various perspectives. Specifically, we obtain a quantitative grasp of the possibilities for variation in measurement results that may arise from preconditions or assumptions made at the time the models were designed. In particular, we assess the possibilities for underestimating required capital and the possible magnitude of such underestimates. Then, in our periodic verification framework, which is described below, we make analyses of how to compensate for such underestimates. We apply our understanding of the possibilities for underestimation to the multiplier derived from back testing, and, if the accuracy of the quantification model deteriorates, we introduce a framework for making adjustments in the multiplier to avoid underestimating the amount of required capital.

Figure 3: Bundled sample page used by the final demo.

YOLO Layout Detection

The detector identifies the main semantic regions on the page. Orange regions represent normal text blocks, green regions represent section headers, gray regions represent page header/footer material, and other colors mark additional classes.

Figure 4: Detected layout regions on the bundled sample page.

Reading Order

After OCR, the regions are sorted into a top-to-bottom reading order with column-aware geometric grouping. The order numbers in the following image correspond to the sequence used for Markdown rendering.

Figure 5: Reading order assigned to the detected regions.

Markdown Reconstruction

The reconstructed Markdown preserves page metadata as comments, converts detected section headers to Markdown headings, and renders paragraph regions as normal text. A representative excerpt from the bundled sample is shown below.

```
## (B) Estimation of Loss Frequency Distribution
The Poisson distribution is used for generating the loss frequency distribution. To
estimate the Poisson distribution, it is necessary to estimate the average number of
annual losses, but in this model, we estimate the annual average number of loss cases
for each fiscal year individually.

## (D) Calculation of Required Capital
We calculate required capital by multiplying the 99.9% VaR calculated in the previous
section by the multiplier for each loss event type.
```

4) Implementation

Create Training Data

The dataset conversion scripts transform the DocLayNet-style COCO annotations into YOLO detection and segmentation datasets. The detection dataset stores rectangular boxes, while the segmentation dataset stores polygon masks when available. Both conversions preserve the same semantic class list so that model outputs map directly into Markdown rendering rules.

Train and Evaluate Models

Training is handled through small Python entrypoints around Ultralytics YOLO. The final detection run used YOLOv26m at 1024 pixel resolution, batch size 8, cosine learning rate scheduling, and conservative document augmentations. Validation was then run on the saved best checkpoint to produce precision, recall, mAP50, mAP50-95, PR curves, and a normalized confusion matrix.

OCR and Reconstruction

The OCR stage uses the packaged PP-OCRv6 medium recognition model. Since this is a recognizer, not a detector, wide YOLO crops are split into text lines and chunks before recognition. For table-like crops, long horizontal and vertical ruling lines are removed before row splitting so that table borders do not merge unrelated text rows. Markdown rendering then uses region class names, OCR text, and OCR line geometry to choose headings, paragraphs, lists, tables, figure placeholders, and page metadata comments.

Reproducible Demo

The repository includes one Windows script and one Linux/macOS script. Each script installs `uv` if needed, runs `uv sync`, and starts the Streamlit demo. The final YOLO checkpoints and OCR model are included in the repository, so the demo does not require external model downloads during grading.

5) Code

The project code is available at:

https://github.com/austinconnor/cap6665-project-1